In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages

np.set_printoptions(legacy='1.25') #removes the funny printing
```

Pandas

Pictures are for fun. No need to add them to your notebook!

In [3]:
```
from IPython.display import Image
Image(filename='PeterPanda2-phineas-pherb.jpg',width=400)
```

Out[3]:



Automated Pandas?

In [4]:
```
from IPython.display import Image
Image(filename='Pandaborg.jpg',width=400)
```

Out[4]:



Today we want to automate a few things in pandas. this means we will need to clean up our coding and call things in a better method. I will walk you through it. Lets read in our file and get started. We will use well_data.csv again. Look on courseworks and you will see our goal is making a file with the graphs we want. I kept the df in the name as it is a dataframe

In [9]:
```
df=pd.read_csv('well_data.csv')
```

Lets print the arsenic data again

In [10]:
```
df['As'].head()
```

Out[10]:
```
0         NaN
1         NaN
2         NaN
3    78.97747
4         NaN
Name: As, dtype: float64
```

Now instead of using 'As' lets set i to 'As' and try and call it. **Remember** I am using head() to save space printing. You do not need it for class.

In [11]:
```
i='As'
df[i].head()
```

Out[11]:
```
0         NaN
1         NaN
2         NaN
3    78.97747
4         NaN
Name: As, dtype: float64
```

We can also do fucntions on it. Lets call dtype which tells us the data type.

In [12]:
```
i='As'
df[i].dtype
```

Out[12]:
```
dtype('float64')
```

now try setting i to Drink

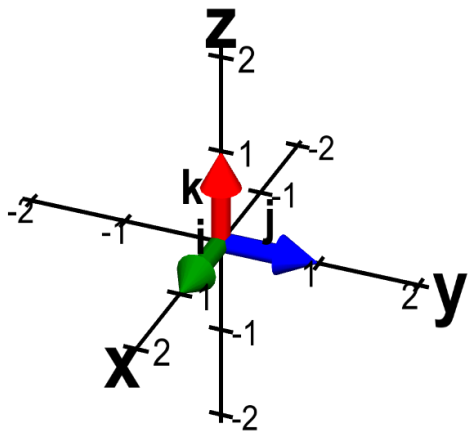
In [9]:
```

```

Out[9]:
```
dtype('O')
```

In [8]:
```
from IPython.display import Image
Image(filename='standard_unit_vectors_3d.png',width=400)
#copied from http://mathinsight.org/cross_product_formula
```

Out[8]:



I am not going to go into it. But i,j,k are used for vectors in math and are thus the common names people iterate over. So i,j,k is the vector in x,y,z space. But you could call it anything. But i,j,k are simple and easy

In [13]:
```
I_need_to_think_of_a_better_name='As'
df[I_need_to_think_of_a_better_name]
```

Out[13]:
```
0           NaN
1           NaN
2           NaN
3       78.97747
4           NaN
          ...
725      0.13000
726     17.39000
727    112.37000
728    248.93000
729          NaN
Name: As, Length: 730, dtype: float64
```

now what if we wanted to print the datatypes one by one. we could use a for loop. The nice thing about Pandas is you can iterate over the columns automatically.

In [14]:
```
for i in df:
    print (i)
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

I used to call it col for i but this year I am going to try column

In [15]:
```
for column in df:
    print (column)
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Now that you know how to iterate can you go through and print each data type with nice? I am switchin to use f-string formatting.

In [43]:
```

```

```
Well_ID is of type int64
Lat     is of type float64
Lon     is of type float64
Depth   is of type int64
Drink   is of type object
Si      is of type float64
P       is of type float64
S       is of type float64
Ca      is of type float64
Fe      is of type float64
Ba      is of type float64
Na      is of type float64
Mg      is of type float64
K       is of type float64
Mn      is of type float64
As      is of type float64
Sr      is of type float64
F       is of type float64
Cl      is of type float64
SO4     is of type float64
Br      is of type float64
```

Graphing a variable

Now lets get our code for printing a graph and instead of saying As. Lets put in a variable. Bur First this is my original simple HW answer.

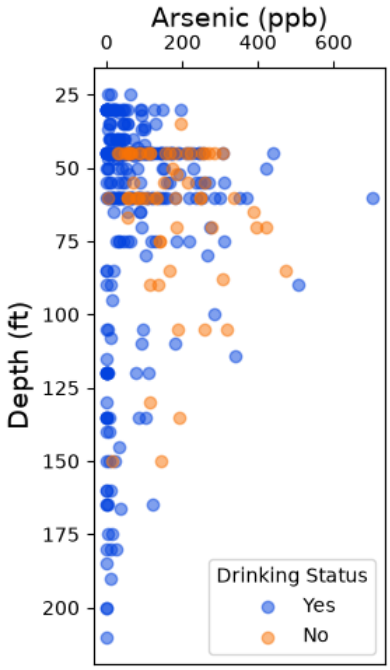
In [92]:
```
fig,ax=plt.subplots(layout='tight', figsize=(3,5))

ax.scatter(df[df.Drink=='Y']['As']
           ,df[df.Drink=='Y']['Depth'],
           alpha=0.5,
           label='Yes'
           ,color='xkcd:blue')

ax.scatter(df[df.Drink=='N']['As']
           ,df[df.Drink=='N']['Depth']
           ,alpha=0.5
           ,label='No'
           ,color='xkcd:orange')

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel('Arsenic (ppb)',fontsize=14)
ax.set_ylabel('Depth (ft)',fontsize=14)

ax.legend(loc='best',title='Drinking Status')
plt.show()
```



Change to a variable for arsenic

1. Make a new variable and set to As. I called it Xcolumn='As'
2. Subsitute your variable in the graphs and in the axis title
3. run the script.

In [94]:
```
Xcolumn='As'

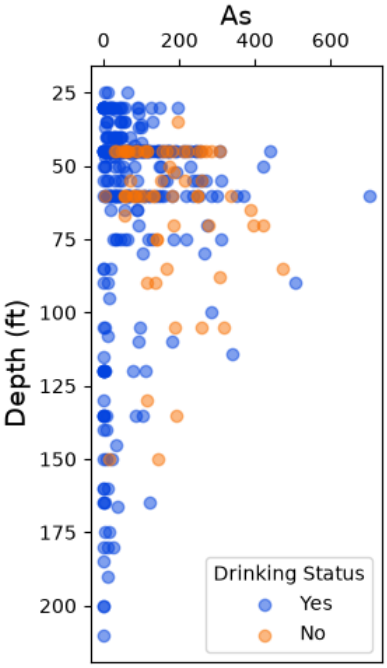
fig,ax=plt.subplots(layout='tight',figsize=(3,5))

ax.scatter(df[df.Drink=='Y'][Xcolumn]
           ,df[df.Drink=='Y']['Depth'],
           alpha=0.5,
           label='Yes'
           ,color='xkcd:blue')

ax.scatter(df[df.Drink=='N'][Xcolumn]
           ,df[df.Drink=='N']['Depth']
           ,alpha=0.5
           ,label='No'
           ,color='xkcd:orange')

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(Xcolumn,fontsize=14)
ax.set_ylabel('Depth (ft)',fontsize=14)

ax.legend(loc='best',title='Drinking Status')
plt.show()
```



Now change the variable

- Now go back and change the variable name to a new column
- How did it go?

Saving a figure

To save just one figure you can right click on an image. But even easier is adding a line of code. I always add this line

fig.savefig('Name_of_figure.jpg',dpi=200,bbox_inches='tight')

- name the figure and the ending sets the type. jpg,gif,pdf.....

- dpi is the dots per inch and is the resolution. For a fancy figure increase to 600 or so.
- bbox_inches='tight' is this weird key word you need. If you don't have this and you have a legend, the legend gets cut off.
- It just puts the file in your working directory. Try it.

In [111…
```
Xcolumn='As'

fig,ax=plt.subplots(layout='tight',figsize=(3,4))

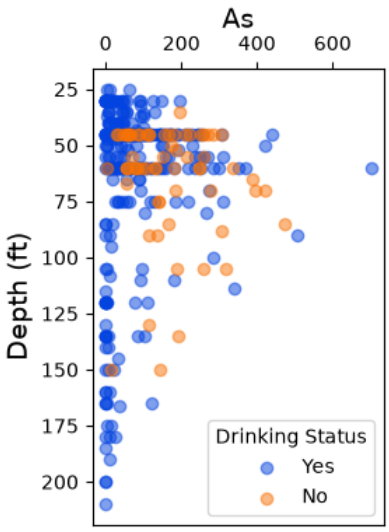

ax.scatter(df[df.Drink=='Y'][Xcolumn]
           ,df[df.Drink=='Y']['Depth'],
           alpha=0.5,
           label='Yes'
           ,color='xkcd:blue')

ax.scatter(df[df.Drink=='N'][Xcolumn]
           ,df[df.Drink=='N']['Depth']
           ,alpha=0.5
           ,label='No'
           ,color='xkcd:orange')

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(Xcolumn,fontsize=14)
ax.set_ylabel('Depth (ft)',fontsize=14)

ax.legend(loc='best',title='Drinking Status')
plt.show()

fig.savefig('First_figure.jpg',dpi=200,bbox_inches='tight')
```



Better figure naming

- since we have a variable that we call for plotting we can also use that variable for naming.
- take the variable and add .jpg to the end
- We can use the fancier and more modern f-string formatting
- make a new variable that is the filename
- try a few columns and see what happens

In [96]:
```
Xcolumn='Fe'
filename=f'{Xcolumn}.jpg'

fig,ax=plt.subplots(layout='tight',figsize=(3,5))

ax.scatter(df[df.Drink=='Y'][Xcolumn]
           ,df[df.Drink=='Y']['Depth'],
           alpha=0.5,
           label='Yes'
           ,color='xkcd:blue')

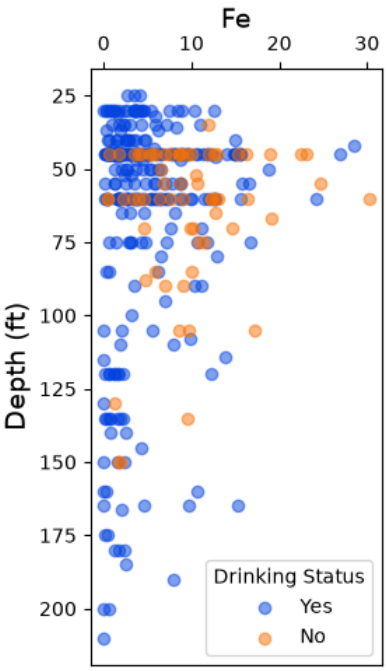
ax.scatter(df[df.Drink=='N'][Xcolumn]
           ,df[df.Drink=='N']['Depth']
```

```
           ,alpha=0.5
           ,label='No'
           ,color='xkcd:orange')

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(Xcolumn,fontsize=14)
ax.set_ylabel('Depth (ft)',fontsize=14)

ax.legend(loc='best',title='Drinking Status')
plt.show()

fig.savefig(filename,dpi=200,bbox_inches='tight')
```



Easy? I Was the Easy Part!

If you forget the column names...

In [21]:
```
df.columns
```

Out[21]:
```
Index(['Well_ID', 'Lat', 'Lon', 'Depth', 'Drink', 'Si', 'P', 'S', 'Ca', 'Fe',
       'Ba', 'Na', 'Mg', 'K', 'Mn', 'As', 'Sr', 'F', 'Cl', 'SO4', 'Br'],
      dtype='object')
```

Folder Mess

- Once you start making a lot of files you need to organize!
- Your working folder will become a mess.
- Here is a trick
- Make a folder called Results
  - Make it on your computer. You can do it in your folders or hit the foler+ symbol to the left
  - Make it in your current working directory
- Now in the name of your file add the prefix Results/
- Now the file will get dumped into that folder. Try it.

In [97]:
```
Xcolumn='Fe'
filename=f'Results/{Xcolumn}.jpg'

fig,ax=plt.subplots(layout='tight',figsize=(3,5))

ax.scatter(df[df.Drink=='Y'][Xcolumn]
           ,df[df.Drink=='Y']['Depth'],
           alpha=0.5,
           label='Yes'
```

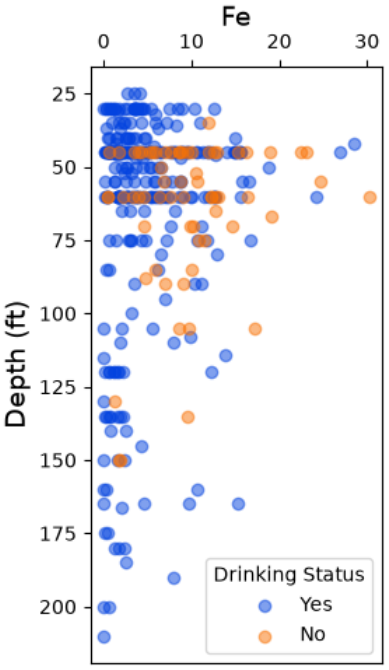
```
            ,color='xkcd:blue')

ax.scatter(df[df.Drink=='N'][Xcolumn]
           ,df[df.Drink=='N']['Depth']
           ,alpha=0.5
           ,label='No'
           ,color='xkcd:orange')

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(Xcolumn,fontsize=14)
ax.set_ylabel('Depth (ft)',fontsize=14)

ax.legend(loc='best',title='Drinking Status')
plt.show()

fig.savefig(filename,dpi=200,bbox_inches='tight')
```



Goal

A plot of Depth versus every parameter.

- Combine our for loop with our plotting with a variable.
- I added a line to drop the columns we don't want to print.
- could it be easy?
- Lets go back to our for loop
- Make a new list of columns where we drop the ones we don't want to plot
- Does this give our columns?

In [98]:
```
columns_to_plot = df.drop(columns=["Lat", "Lon", "Depth", "Drink","Well_ID"]).columns

for Xcolumn in columns_to_plot :
    print (Xcolumn)
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

You are Ready

- Just put your plotting code insde the for loop!!!!!

BUT WE HAVE ONE ISSUE!

- That would call fig,ax=plt.subplots() 16 times!!!!!
- That will kill all the memory on your computer
- So you need to close the fig after each for loop.
- add plt.close(fig) at the end of the foor loop
- Also, no need to show the plot. So remove plt.show()
- You won't see the graphs but they will be saved! Go look in the folder
- Make sure to delete the line XColumn="Fe" or similar if you are only getting one plot.

In [99]:
```
columns_to_plot = df.drop(columns=["Lat", "Lon", "Depth", "Drink","Well_ID"]).columns

for Xcolumn in columns_to_plot :
    print (Xcolumn)

    filename=f'Results/{Xcolumn}.jpg'

    fig,ax=plt.subplots(layout='tight',figsize=(3,5))

    ax.scatter(df[df.Drink=='Y'][Xcolumn]
               ,df[df.Drink=='Y']['Depth'],
               alpha=0.5,
               label='Yes'
               ,color='xkcd:blue')

    ax.scatter(df[df.Drink=='N'][Xcolumn]
               ,df[df.Drink=='N']['Depth']
               ,alpha=0.5
               ,label='No'
               ,color='xkcd:orange')

    ax.invert_yaxis()
    ax.xaxis.set_label_position('top')
    ax.xaxis.tick_top()
    ax.set_xlabel(Xcolumn,fontsize=14)
    ax.set_ylabel('Depth (ft)',fontsize=14)

    ax.legend(loc='best',title='Drinking Status')

    fig.savefig(filename,dpi=200,bbox_inches='tight')
    plt.close(fig) #closes the fig each time through to save space.

print('Yeah out of for loop and all printed!')
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
Yeah out of for loop and all printed!
```

Amazing!

That was amazing! Did you look in your folder and now see all those files you made?

Look at them?

Remember nothing is supposed to show up on your screen!

PDF

- We could also make a multi-page pdf instead.
- Make a variable that is the name of the pdf file. It is called pdf_path.
- add a "with" statement. with PdfPages(pdf_path) as pdf:
  - with is fancy python. It says to work with PdfPages to open the pdf file
  - then no matter what goes right or wrong it closes it at the end
  - we should have no messed up memory
- indent all of your earlier code
- now save the fig to the pdf. pdf.savefig does it!

In [100…

```python
pdf_path = 'Results/OneBigPDF.pdf' #Name a pdf file

columns_to_plot = df.drop(columns=["Lat", "Lon", "Depth", "Drink","Well_ID"]).columns

with PdfPages(pdf_path) as pdf:
    for Xcolumn in columns_to_plot :
        print (Xcolumn)

        #filename=f'Results/{Xcolumn}.jpg' #No need to have a filename!

        fig,ax=plt.subplots(layout='tight',figsize=(3,5))

        ax.scatter(df[df.Drink=='Y'][Xcolumn]
                   ,df[df.Drink=='Y']['Depth'],
                   alpha=0.5,
                   label='Yes'
                   ,color='xkcd:blue')

        ax.scatter(df[df.Drink=='N'][Xcolumn]
                   ,df[df.Drink=='N']['Depth']
                   ,alpha=0.5
                   ,label='No'
                   ,color='xkcd:orange')

        ax.invert_yaxis()
        ax.xaxis.set_label_position('top')
        ax.xaxis.tick_top()
        ax.set_xlabel(Xcolumn,fontsize=14)
        ax.set_ylabel('Depth (ft)',fontsize=14)

        ax.legend(loc='best',title='Drinking Status')

        pdf.savefig(dpi=200,bbox_inches='tight')  # save to a file

        plt.close(fig) #closes the fig each time through to save space.

print(f'Successfully exported {len(columns_to_plot)} pages to {pdf_path}!')
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
Successfully exported 16 pages to Results/OneBigPDF.pdf!
```

You are now an expert

You should now be an expert on make figures and saving them!

In [ ]:

In [ ]:

Subplots!!!!!!

Lets line up plots next to each other!

- Now after looking at all the data I realize I want to plot As, Fe, S on 3 plots next to each other. lets see if we can do this.
- here is an example. http://matplotlib.org/examples/pylab_examples/subplots_demo.html
- So if you look I want to share a y-axis. Lets try..... Lets see if we can learn. I am nervous as these plot always give me trouble
- so I going to start simple and just plot versus depth

Subplots

- In subplots we will get three sets of axes. One for each plot.
- Also, I am telling it to give me 1 row by 3 columns. then we are going to share the y-axes.
- Also since we have 3 plots on one page we have 1 fig and 3 ax values.

In [101…

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey=True)

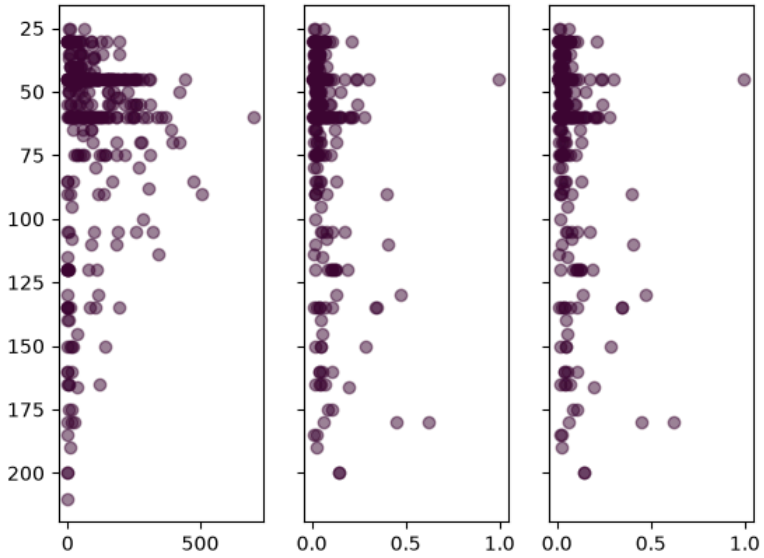
Xcolumn='As'
ax1.scatter(df[Xcolumn]
            ,df['Depth']
            ,alpha=0.5
           ,color='xkcd:aubergine')

Xcolumn='Fe'
ax2.scatter(df[column]
            ,df['Depth']
            ,alpha=0.5
           ,color='xkcd:aubergine')

Xcolumn='S'
ax3.scatter(df[column]
            ,df['Depth']
            ,alpha=0.5
           ,color='xkcd:aubergine')

ax1.invert_yaxis()

plt.show()
```



ax1, ax2, ax3 is clumsy

- First step is I am going to make my ax into a list/array.
- Imagine we wanted 2, 5, or 10 plots. We don't want to write ax10 etc. We can make an array so we can loop over it.

In [102…

```
fig, ax = plt.subplots(1, 3, sharey=True)

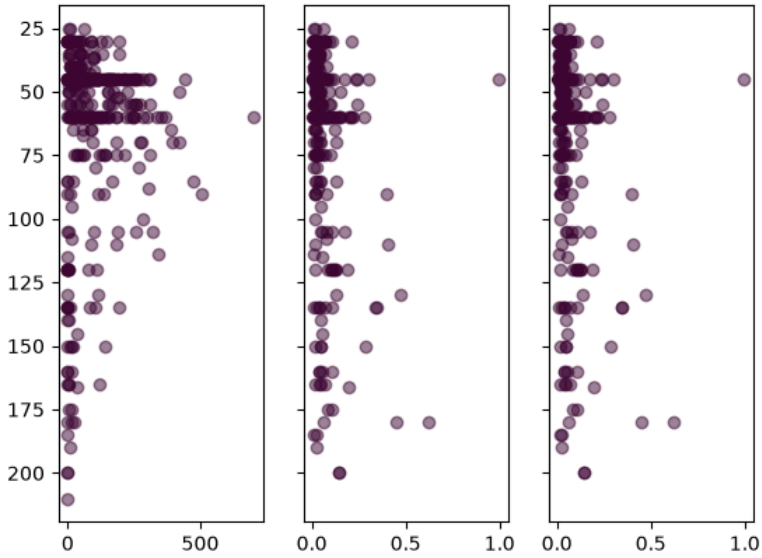
Xcolumn='As'
ax[0].scatter(df[Xcolumn]
              ,df['Depth']
              ,alpha=0.5
             ,color='xkcd:aubergine')

Xcolumn='Fe'
ax[1].scatter(df[column]
              ,df['Depth']
              ,alpha=0.5
             ,color='xkcd:aubergine')

Xcolumn='S'
ax[2].scatter(df[column]
              ,df['Depth']
              ,alpha=0.5
             ,color='xkcd:aubergine')

ax[0].invert_yaxis()

plt.show()
```



That is really encouraging. But we have to get rid of the hardwires and do it in a for loop eventually. This way we could just choose our parameters and it would do it. So lets go back and do for loops.

In [103…
```
columns_to_plot=['As','Fe','S']
for Xcolumn in columns_to_plot:
    print(Xcolumn)
```

```
As
Fe
S
```

Flashback to for loops. This is what I wrote in the notes!

Python has a simple trick to count through a list and give you the list and the count. It may not make sense now but it will later in the semester!

This is the example I gave.

In [104…
```
mystrlist=['env','chem','bio','psych']
for i,mystr in enumerate(mystrlist):
    print (i,mystr)
```

```
0 env
1 chem
2 bio
3 psych
```

Now lets do it for our columns_to_plot!

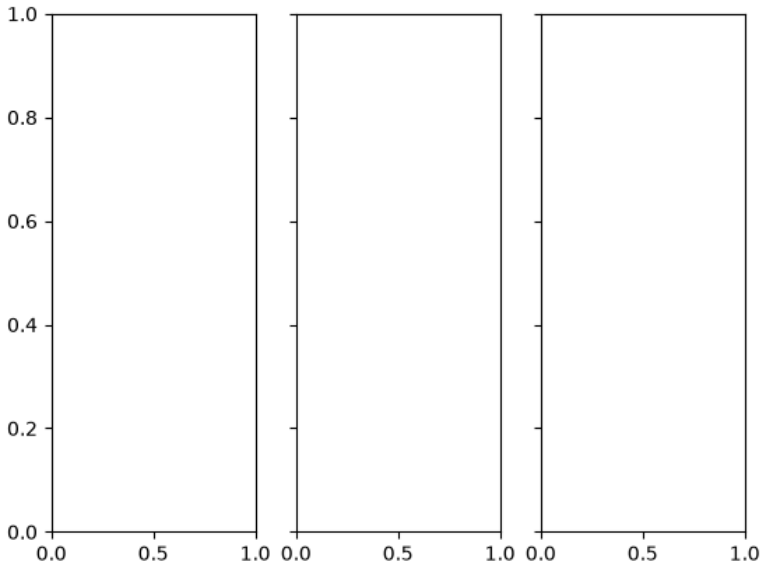
In [105…
```
columns_to_plot=['As','Fe','S']
for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)
```

```
0 As
1 Fe
2 S
```

Lets keep getting rid of hard wires. Now instead of doing subplots(1,3) lets use the length of columns_to_plot!

In [106…
```
columns_to_plot=['As','Fe','S']
fig, ax = plt.subplots(1, len(columns_to_plot), sharey=True)

plt.show()
```



Now put it togehter!

- think about what needs to be in the foor loop
- What should be before?
- after?

In [107…

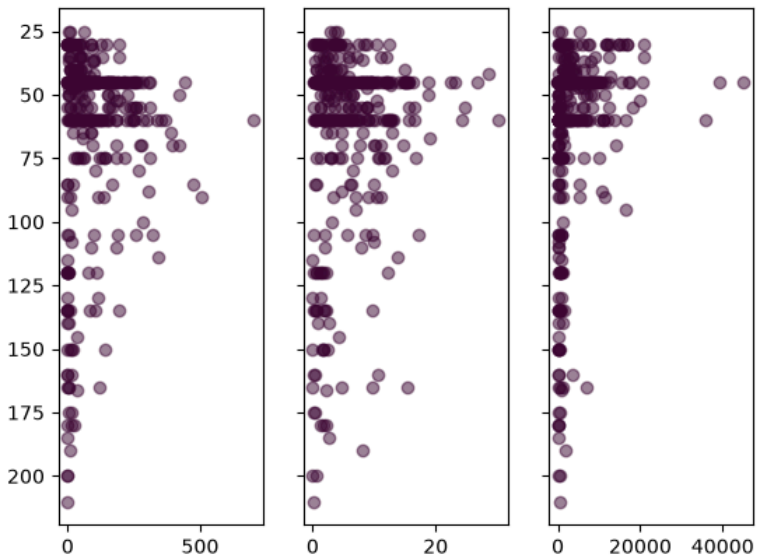
```
columns_to_plot=['As','Fe','S']
fig, ax = plt.subplots(1, len(columns_to_plot), sharey=True)

for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)
    ax[count].scatter(df[Xcolumn]
            ,df['Depth']
            ,alpha=0.5
           ,color='xkcd:aubergine')

ax[0].invert_yaxis()

plt.show()
```

```
0 As
1 Fe
2 S
```



Let's clean it up!

- This is looking great. Lets see if we can clean it up.
- label the y axis.

- make the whole thing bigger!
- make 4 ticks on the x-axis
  - ax.locator_params(nbins=4,axis='x') to set the number of ticks on the x axis.

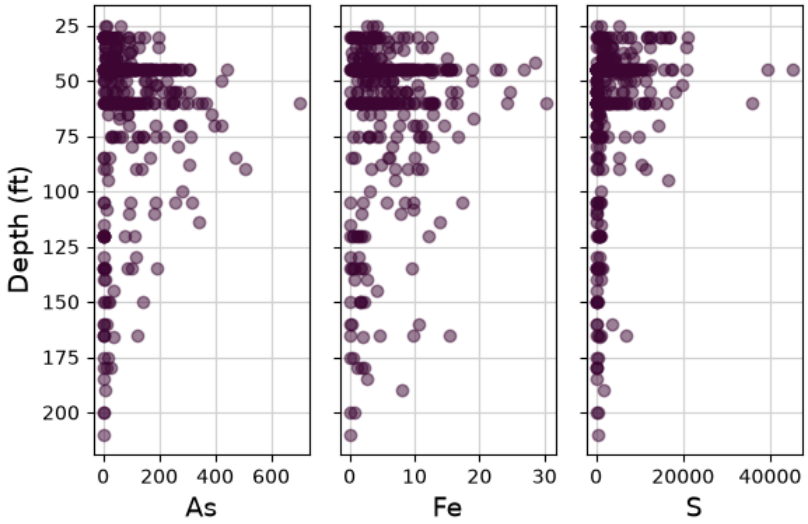
In [108…
```
columns_to_plot=['As','Fe','S']
fig, ax = plt.subplots(1, len(columns_to_plot)
                       , sharey=True
                       , figsize=(6, 4)
                       , layout='tight')

for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)
    ax[count].scatter(df[Xcolumn]
                      ,df['Depth']
                      ,alpha=0.5
                      ,color='xkcd:aubergine'
                      ,zorder=10)  #the highest zorder is drawn last
    ax[count].set_xlabel(Xcolumn,fontsize=14) #labels with the element
    ax[count].locator_params(nbins=4,axis='x')  #gives us 4 ticks marks on the x axies
    ax[count].grid(True, linestyle='-', color='lightgrey',zorder=0)

ax[0].invert_yaxis()
ax[0].set_ylabel('Depth (ft)',fontsize=14)

plt.show()
```

```
0 As
1 Fe
2 S
```



That appears nicer! But beauty is in the eye of the beholder. You can choose what you think makes a nice and professional graph

If you want to publish it you need to label your graphs a,b,c, etc.

We will add these the same way we added the regression properties but instead we are going to add a letter. This is going to get weird to convert a number to a letter.

1. define your props-properties for your bbox-bounding box.
2. define your letter. this is weird. You are going to take the ascii number that is A. ord('A') then add our counter to it and turn it back to a letter. Adding a 0 is A, adding a 1 is a B.
3. Then we will place our letters ising ax.text. Remember we have to transfrom from absolute axis to percent axis. That is what transform does.
4. choose if you want the axes on top or bottom
5. zorder tells what order to plot. so 10 is above 1.

In [109…
```
props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)

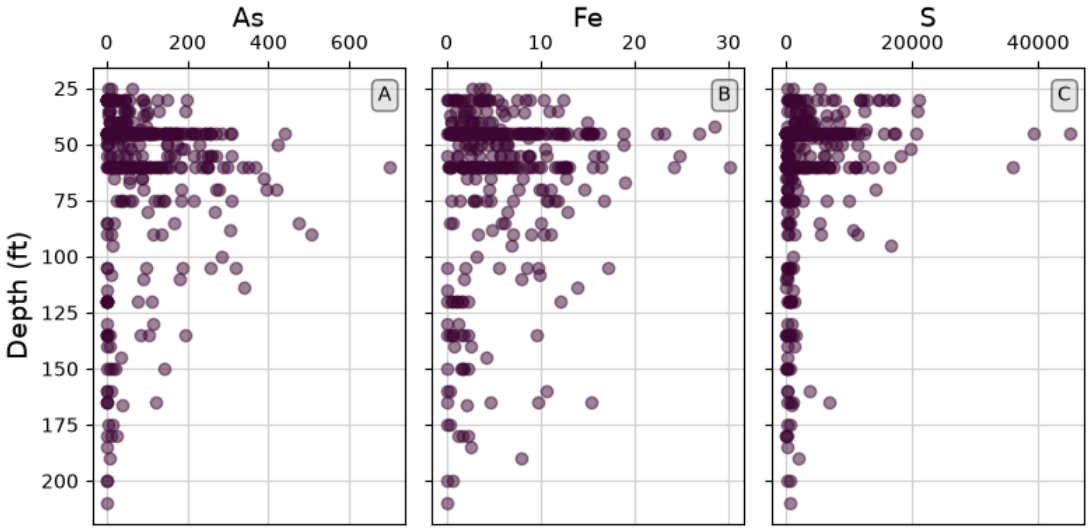
columns_to_plot=['As','Fe','S']
fig, ax = plt.subplots(1, len(columns_to_plot)
                       , sharey=True
                       , figsize=(8, 4)
                       , layout='tight')
```

```
for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)
    ax[count].scatter(df[Xcolumn]
                      ,df['Depth']
                      ,alpha=0.5
                      ,color='xkcd:aubergine'
                      ,zorder=10)
    ax[count].set_xlabel(Xcolumn,fontsize=14) #labels with the element
    ax[count].xaxis.tick_top()      # chooose if you want on top
    ax[count].xaxis.set_label_position('top')  #choose if you want on top
    ax[count].locator_params(nbins=4,axis='x')  #gives us 4 ticks marks on the x axies
    ax[count].grid(True, linestyle='-', color='lightgrey',zorder=0)
    ax[count].text(0.96,0.96,chr(count + ord('A'))
                   ,transform=ax[count].transAxes\
                   ,fontsize=10
                   ,verticalalignment='top'
                   ,horizontalalignment='right'
                   ,bbox=props)

ax[0].invert_yaxis()
ax[0].set_ylabel('Depth (ft)',fontsize=14)

plt.show()
```

```
0 As
1 Fe
2 S
```

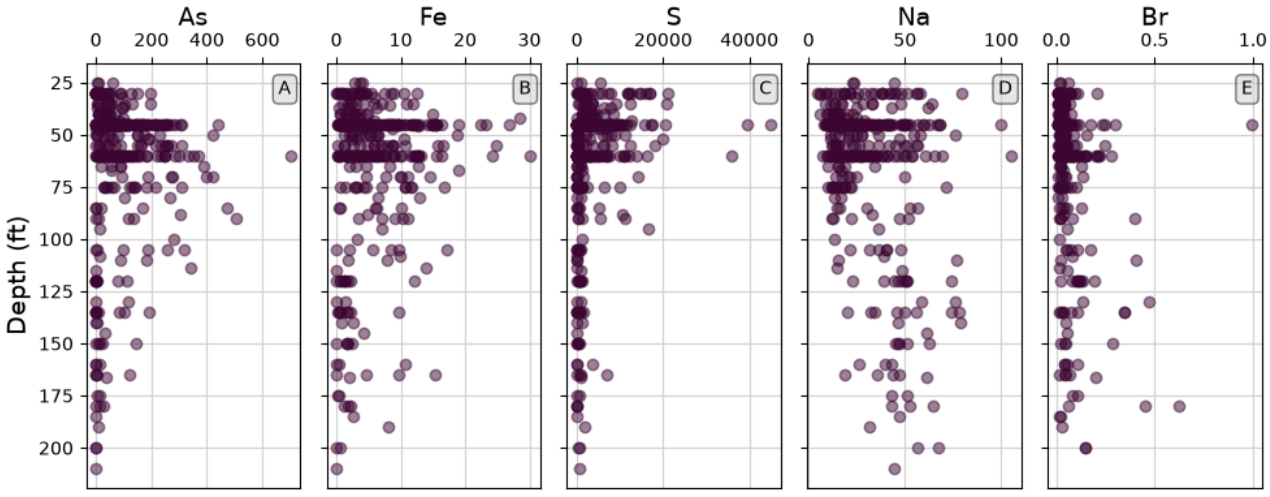


Can you change it to 5 elements?

In [83]:

```
0 As
1 Fe
2 S
3 Na
4 Br
```

Out[83]: Text(0, 0.5, 'Depth (ft)')

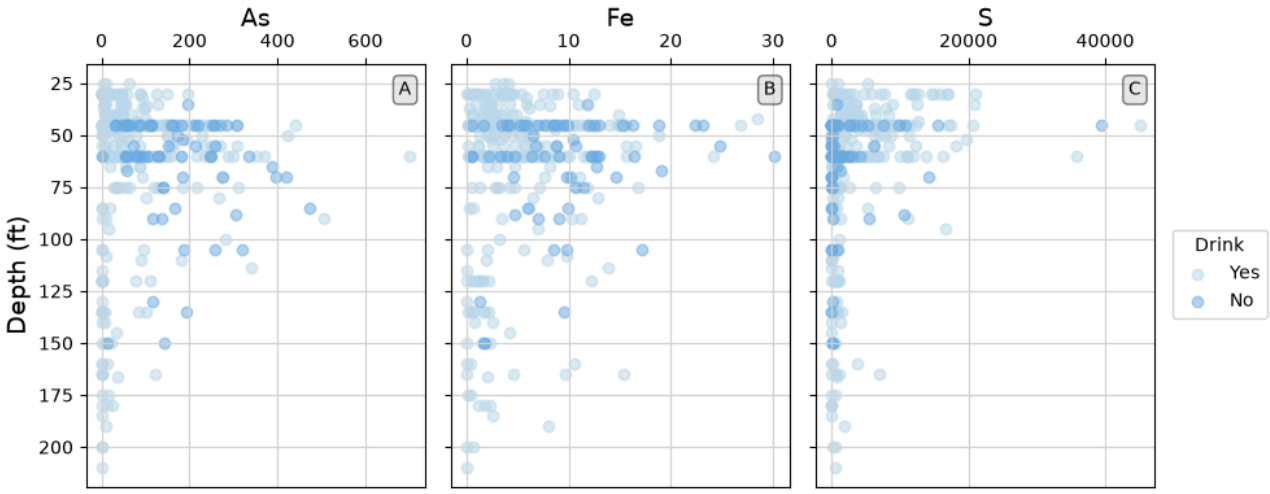


Bonus Fun.

- color the points by Drink
- add a legend
- color by columbia blues https://en.wikipedia.org/wiki/Columbia_blue

In [87]:

```
0 As
1 Fe
2 S
```



Homework Hints

The Drink column might cause a crash for plotting. Remember how we excluded columns

Answers

In [10]:

```
i='Drink'
df[i].dtype
```

Out[10]: dtype('O')

In [19]:

```
for col in df:
    print(f'{col:7s} is of type  {df[col].dtype}')
```

```
Well_ID is of type  int64
Lat     is of type  float64
Lon     is of type  float64
Depth   is of type  int64
Drink   is of type  str
Si      is of type  float64
P       is of type  float64
S       is of type  float64
Ca      is of type  float64
Fe      is of type  float64
Ba      is of type  float64
Na      is of type  float64
Mg      is of type  float64
K       is of type  float64
Mn      is of type  float64
As      is of type  float64
Sr      is of type  float64
F       is of type  float64
Cl      is of type  float64
SO4     is of type  float64
Br      is of type  float64
```

In [84]:
```
props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)

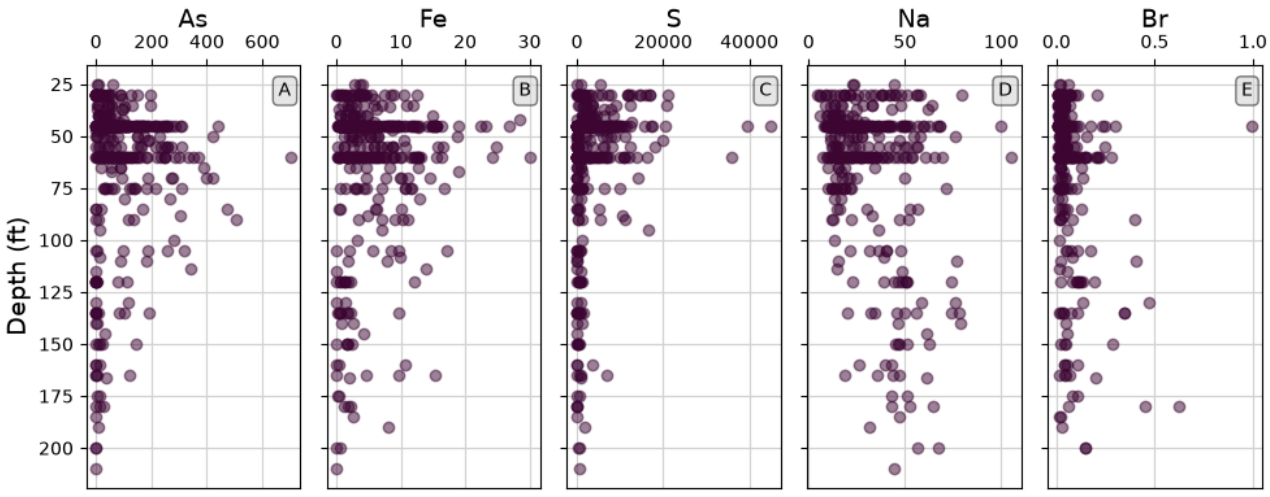
columns_to_plot=['As','Fe','S','Na','Br']
fig, ax = plt.subplots(1, len(columns_to_plot), sharey=True, figsize=(8, 4), layout='tight')

for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)
    ax[count].scatter(df[Xcolumn]
                      ,df['Depth']
                      ,alpha=0.5
                      ,color='xkcd:aubergine'
                      ,zorder=10)
    ax[count].set_xlabel(Xcolumn,fontsize=14) #labels with the element
    ax[count].xaxis.tick_top()      # chooose if you want on top
    ax[count].xaxis.set_label_position('top')  #choose if you want on top
    ax[count].locator_params(nbins=4,axis='x')  #gives us 4 ticks marks on the x axies
    ax[count].grid(True, linestyle='-', color='lightgrey',zorder=0)
    ax[count].text(0.96,0.96,chr(count + ord('A'))
                   ,transform=ax[count].transAxes\
                   ,fontsize=10
                   ,verticalalignment='top'
                   ,horizontalalignment='right'
                   ,bbox=props)

ax[0].invert_yaxis()
ax[0].set_ylabel('Depth (ft)',fontsize=14)
```

```
0 As
1 Fe
2 S
3 Na
4 Br
```

Out[84]: Text(0, 0.5, 'Depth (ft)')



In [110…
```
props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)
```

```
columns_to_plot=['As','Fe','S']
fig, ax = plt.subplots(1, len(columns_to_plot), sharey=True, figsize=(6, 4), layout='tight')

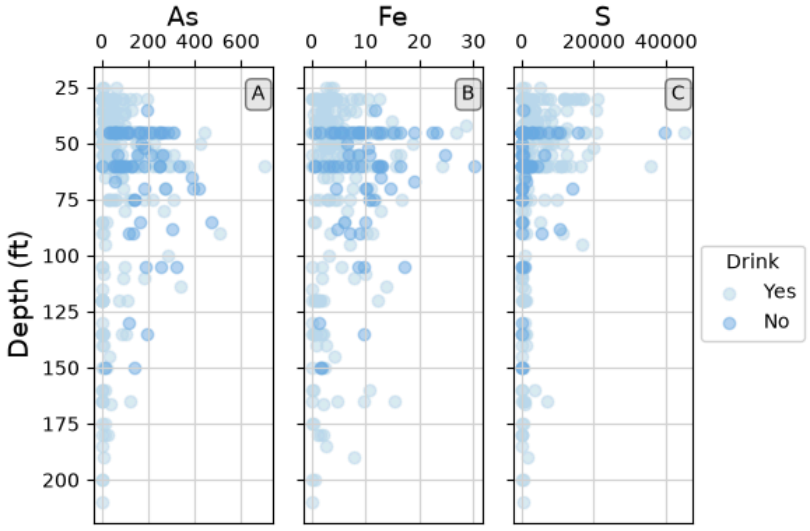
for count,Xcolumn in enumerate(columns_to_plot):
    print(count,Xcolumn)

    ax[count].scatter(df[df.Drink=='Y'][Xcolumn]
                      ,df[df.Drink=='Y'].Depth
                     ,alpha=0.5
                      ,label='Yes'
                     ,color='#B9D9EB')
    ax[count].scatter(df[df.Drink=='N'][Xcolumn]
                      ,df[df.Drink=='N'].Depth
                      ,alpha=0.5,
                      label='No'
                      ,color='#6CACE4')

    ax[count].set_xlabel(Xcolumn,fontsize=14) #labels with the element
    ax[count].xaxis.tick_top()      # chooose if you want on top
    ax[count].xaxis.set_label_position('top')  #choose if you want on top
    ax[count].locator_params(nbins=4,axis='x')  #gives us 4 ticks marks on the x axies
    ax[count].grid(True, linestyle='-', color='lightgrey',zorder=0)
    ax[count].text(0.96,0.96,chr(count + ord('A'))
                   ,transform=ax[count].transAxes\
                   ,fontsize=10
                   ,verticalalignment='top'
                   ,horizontalalignment='right'
                   ,bbox=props)

ax[0].invert_yaxis()
ax[0].set_ylabel('Depth (ft)',fontsize=14)
ax[count].legend(title='Drink',loc=(1.05,0.4))
plt.show()
```

```
0 As
1 Fe
2 S
```



In [ ]:

In [ ]: